Honeypot IDS – Pool test report

Generated: 2026-10-17 16:40:29
Target: ids (local) (https://localhost)
Patterns: A: CVE-2024-2387, CVE-2024-1071; B: CVE-2025-47577, CVE-2025-4403; C: GENERIC-WP-003, GENERIC-WP-002; D: CVE-2024-27956, CVE-2024-1071; E: CVE-2022-0739, CVE-2025-4403; F: CVE-2024-2879, GENERIC-WP-002; G: CVE-2023-2986, CVE-2024-1071; H: CVE-2025-2266, CVE-2025-4403; I: GENERIC-SQLI-001, GENERIC-WP-002; J: GENERIC-XSS-001, CVE-2024-1071
Windows: 10
Session tamper-proofing: 2 clearing(s) and 4 forgery(ies), in random windows

---

Result

EXCLUSIVE **Every user has a honeypot pool of their own**

- User A → pool 1, User B → pool 2, User C → pool 3, User D → pool 4, User E → pool 6, User F → pool 7, User G → pool 5, User H → pool 8, User I → pool 10, User J → pool 9.

What is being tested

Users are told apart by session, not by address. Each test window has its own cookie jar and its own User-Agent and language, so the proxy sees 10 separate users. By default every flagged session is given a honeypot pool **of its own** (an eshop container with its own seeded database): the router hands a new user a free ready pool no other session owns, and the pools scale with the user sessions – a session that finds no free pool borrows one round-robin while pool_manager builds one for it, then moves there. Scaling stops at the configured resource limit (POOL_MAX_MEMORY_MB / POOL_MAX_CPUS / POOL_MAX) or when the host is short of memory, CPU or disk; from then on further users share the ready pools in strict round-robin order.

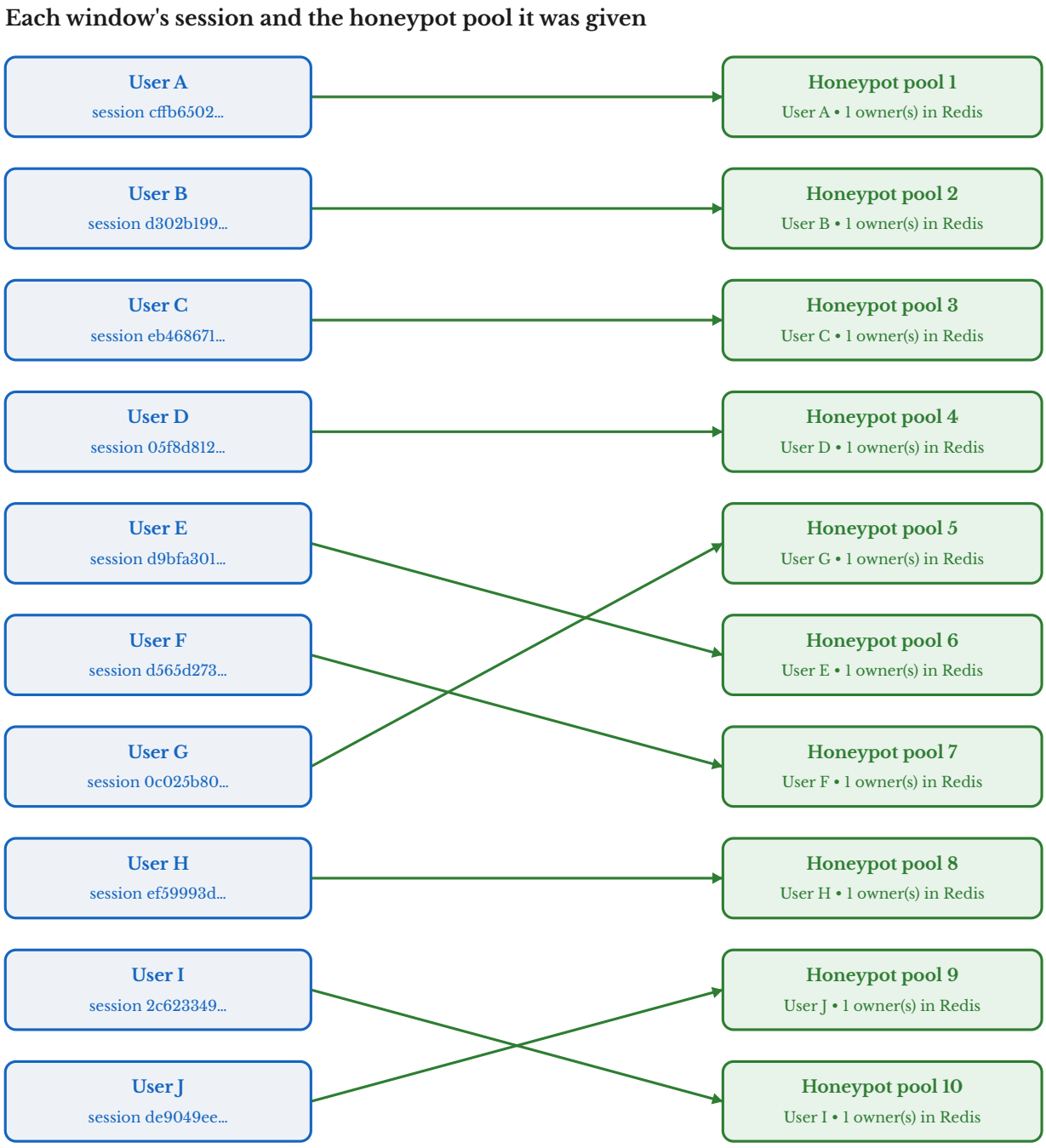


Figure 1. One arrow per test window, from its session to the honeypot pool the router bound it to. Green: the pool belongs to that window alone. Orange: shared under resource pressure (designed). Red: shared or merged without a reason. Grey: not bound.

Windows

Window	Session	Honeypot pool	Language	User-Agent (tail)
User A	cffb6502…	pool 1	en-US,en;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/A
User B	d302b199…	pool 2	de-DE,de;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/B
User C	eb468671…	pool 3	fr-FR,fr;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/C
User D	05f8d812…	pool 4	es-ES,es;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/D
User E	d9bfa301…	pool 6	it-IT,it;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/E
User F	d565d273…	pool 7	nl-NL,nl;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/F
User G	0c025b80…	pool 5	pl-PL,pl;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/G
User H	ef59993d…	pool 8	sv-SE,sv;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/H
User I	2c623349…	pool 10	cs-CZ,cs;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/I
User J	de9049ee…	pool 9	sk-SK,sk;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/J

Session tamper-proofing

Between steps, random windows tampered with their own session. Some **cleared** their data the way a regular visitor does from the browser's settings (every cookie of the site, or all its site data): the stack should still recognise the window by its passive fingerprint (TLS hello, User-Agent, Accept-Language, IP) and keep it on its session. Others **forged** it: their SERVERID cookie was rewritten to an invented session, or to another window's session (a hijack attempt), with a made-up signature. The real cookie is "<session id>.<HMAC>" and only the proxy holds the key, so a forged one must be rejected as not signed by the server – logged in Redis (*session_cookie_rejections*, by the SHA-1 of the value, matched here against the value planted) – and the window recovered into its own session, never the claimed one. A fake session written into localStorage / sessionStorage is never sent to the server, so it must change nothing. A window that lost its cookies also lost WooCommerce's cart cookie, so its cart starts over there. Random seed: 1245065391.

2 clearing(s): 2 kept their session, 0 got a fresh one. 4 forgery(ies): 1 ignored (browser storage), 3 rejected by the proxy. A rejected forgery is scored as tampering and stays on the window's session: User G (+50 before step 5), User C (+50 before step 7), User A (+50 before step 11). That is the raised score under those windows in the step tables from then on; it diverts the window only once its total reaches the threshold.

Time	Step	Window	Tampering	Session before → after	What the stack did
0:30	3	User E	**forged storage** 8 storage entries written	d9bfa301…, production → d9bfa301…, production	ignored same session the server never reads browser storage; kept its session, served by production
0:44	5	User G	**hijack attempt** SERVERID = cffb6502d8f086… (claims session cffb6502 of User A, on pool 1)	0c025b80…, production → 0c025b80…, production	rejected same session rejected (bad_signature, scored +50 as tampering); kept its session, served by production
1:16	7	User C	**forged cookie** SERVERID = 4a2ed29548e7d2… (claims session 4a2ed295)	eb468671…, pool 3 → eb468671…, pool 3	rejected same session rejected (bad_signature, scored +50 as tampering); kept its session, served by pool 3
4:29	8	User J	**cleared site data** 10 cookie(s) (SERVERID, fh_cart + 8 other(s))	de9049ee…, production → de9049ee…, pool 9	same session kept its session, served by pool 9
5:01	10	User D	**cleared site data** 9 cookie(s) (SERVERID, fh_cart + 7 other(s))	05f8d812…, pool 4 → 05f8d812…, pool 4	same session kept its session, served by pool 4
5:15	11	User A	**forged cookie** SERVERID = a1ac408910dac0… (claims session a1ac4089)	cffb6502…, pool 1 → cffb6502…, pool 1	rejected same session rejected (bad_signature, scored +50 as tampering); kept its session, served by pool 1

Step: the tampering happened just before that step of *Step by step*.

Scaling and latency

pool_manager adds one honeypot pool (an eshop container plus its own database seeded from production) for every user session that finds no free pool, up to the resource limit. These figures come from pool_manager's own event log (build phases, hand-overs with their wait time, resource-limit changes) and from the windows themselves (routing read after every step and every 2 s while waiting; the duration of each page load).

Windows	10, of which 7 attacked only after pools were pre-built for them
Run time	6:26 (then 0:53 of clean-up after the reassignment check, not in the figures)
Failed page loads	21 page load(s) did not succeed at the first attempt (28 failed attempt(s)), retried with back-off; every one loaded in the end. Chromium aborts loads in flight (ERR_NETWORK_CHANGED) whenever Docker adds or removes a container's network interface, i.e. while pools are built or removed. 21 of the 21 ran while a pool was being built or removed. By step: Pools pre-built for 7 delayed window(s) 21.
Resource limit	12288 MB, 12 CPUs, POOL_MAX 10 (a pool reserves 768.0 MB / 0.75 CPUs); 10 pool(s) exist, room for 0 more
Pools added during the run	7 built; build time median 48 s, max 53 s
Round-robin while not scaled yet (case A)	0 window(s) borrowed a pool
Round-robin at the resource limit (case B)	0 window(s) share a pool for good; limit first reached at 3:30 (pool limit reached: 10 of POOL_MAX=10 pools)
Forced scale-down at the end	10 → 3 pool(s) in 17 s; pools 1, 2, 3 are declared in docker-compose.yml, only freed, never removed, so more than the spare count (1) remain
Pre-build for the delayed windows	7 free pool(s) ready after 185 s: 6 built for it, 1 spare(s) that were already building (pool 4)

Why it is slow and how to speed it up. A pool is built by cloning production, then re-applying the WordPress storefront config via wp-cli (the “WordPress seed” phase, ~24 s here — the biggest part of a ~53 s build).
That config is identical to production, which was just cloned, so it can be skipped: set **POOL_FAST_CLONE=1** (ids/.env) to inherit the storefront from the clone instead of rebuilding it, saving most of that phase. Verify a built pool still serves the shop before relying on it.
The pre-build ran 2 build(s) at a time, so 7 pools took 4 wave(s). Raise **POOL_PARALLEL_BUILDS** to build them in one wave (the host here has spare cores for it).
Keeping more warm spares (**POOL_SPARES**) means users more often find a pool already built, at the cost of idle resources.

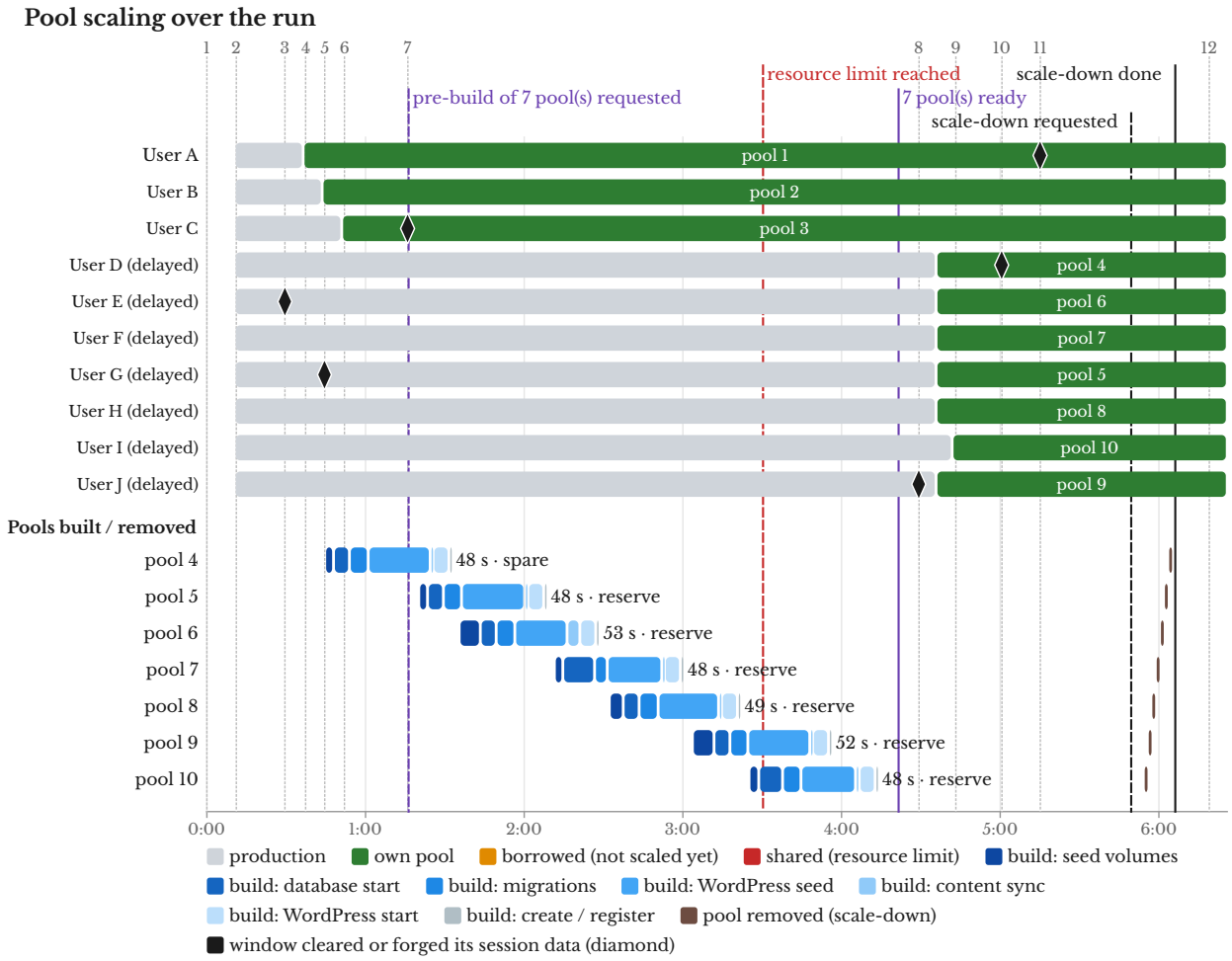


Figure 2. Where each window was served over the run (grey: production, green: its own pool, orange: a borrowed pool while its own was being built, red: shared because the resource limit was reached), and every pool built during the run, its phases shaded dark to light. Dashed grey arrows: a finished pool handed to the window that waited for it. Numbered dotted lines: the steps below. Black diamonds: a window cleared or forged its session data.

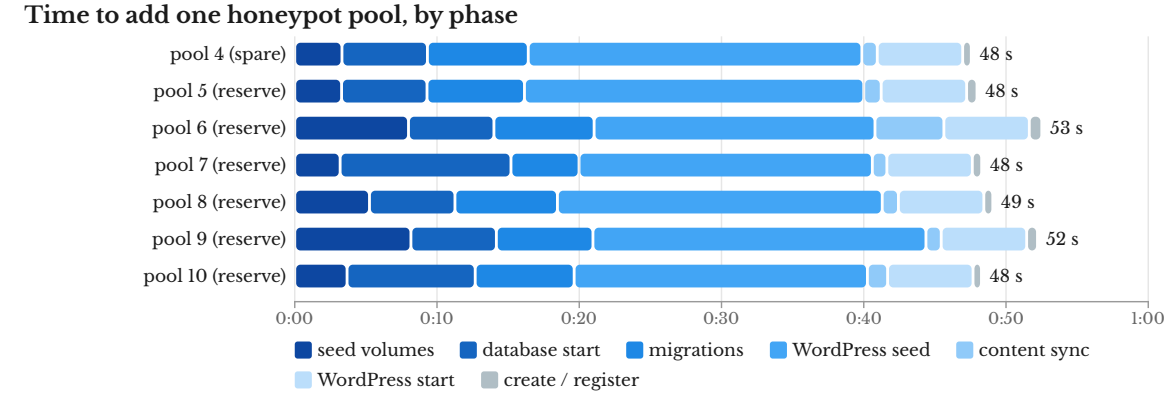


Figure 3. Latency of adding one honeypot pool: wall time from the decision to build it until it was registered healthy, split into pool_manager's phases.

Pool	Built for	Started	Build time	Phases
4	spare	0:45	48 s	seed volumes 3 s, database start 6 s, migrations 7 s, WordPress seed 23 s, content sync 1 s, WordPress start 6 s
5	reserve	1:20	48 s	seed volumes 3 s, database start 6 s, migrations 7 s, WordPress seed 24 s, content sync 1 s, WordPress start 6 s
6	reserve	1:35	53 s	seed volumes 8 s, database start 6 s, migrations 7 s, WordPress seed 20 s, content sync 5 s, WordPress start 6 s
7	reserve	2:11	48 s	seed volumes 3 s, database start 12 s, migrations 5 s, WordPress seed 21 s, content sync 1 s, WordPress start 6 s
8	reserve	2:32	49 s	seed volumes 5 s, database start 6 s, migrations 7 s, WordPress seed 23 s, content sync 1 s, WordPress start 6 s
9	reserve	3:04	52 s	seed volumes 8 s, database start 6 s, migrations 7 s, WordPress seed 23 s, content sync 1 s, WordPress start 6 s
10	reserve	3:25	48 s	seed volumes 4 s, database start 9 s, migrations 7 s, WordPress seed 21 s, content sync 1 s, WordPress start 6 s

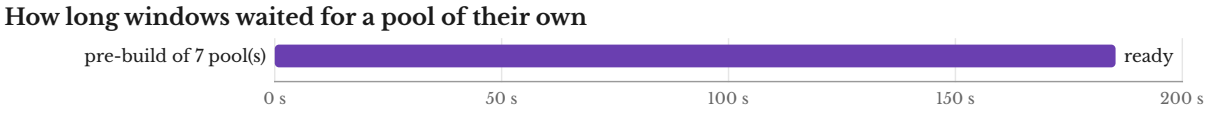


Figure 4. Orange: time a window spent on a borrowed pool (assigned round-robin because no pool was free yet) until it was moved to the pool built for it, as seen by the window (2 s resolution); pool_manager's own measurement in brackets. Red: the window was left on the borrowed pool because the resource limit was reached. Purple: time to pre-build the pools for the delayed windows.

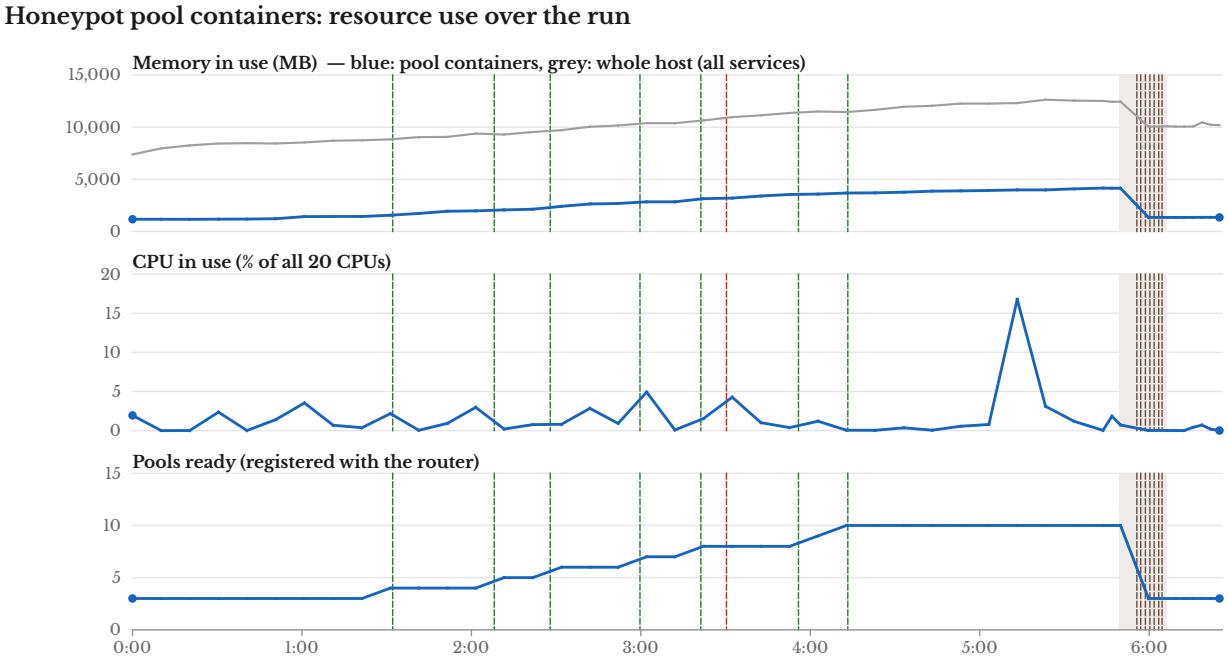


Figure 5. Memory and CPU of the honeypot pool containers (eshop + database) only, and the number of pools registered ready with the router (docker stats every 10 s, every 3 s around the scale-down). CPU is the sum over the pool containers divided by the host's 20 logical processors, so 100% means every core busy. The memory panel's grey line is the whole host's RAM in use (every service, not just the pools). Dashed lines: green a pool finished building, red the resource limit was reached, brown a pool was removed. Shaded: the forced scale-down at the end of the run.

Why pool memory is below the budget

The figure's memory panel counts the honeypot pool containers only. A WordPress pool idles far below both its budget charge and its container limit, so 10 pools use far less than a 12 GB budget suggests; the host as a whole (grey line) is higher because it runs everything else too.

Pool containers, measured peak	4,159 MB across 10 pools (~416 MB per pool) — this is what the figure's blue line shows
Budget charge per pool	768 MB (POOL_RESERVE_MEM_MB) — what the scaling budget counts, deliberately above measured use as headroom
Container memory limit per pool	1536 MB (1024 eshop + 512 database) — the hard ceiling a pool can reach, rarely approached at rest
Whole host RAM	12,630 MB in use of 15,992 MB total (grey line) — includes production, Redis, Suricata and everything else, so it sits well above the pool containers alone

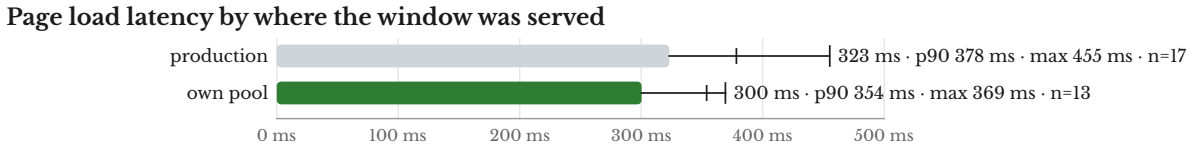


Figure 6. Latency probes: the same product page loaded by one window at a time, three times during the run (all windows on production, after the scale-up, at the end), timed from navigation start to load finished and grouped by where that window was routed right after. Bar: median; short tick: 90th percentile; line end: slowest load.

Round-robin assignment

A diverted session is only ever put on a pool another session uses in two situations, both handled round-robin over the ready pools:

A. Not scaled up yet (no free pool, more can be built)

Not observed: every window found a free pool the moment it was diverted. The delayed windows waited for pre-built pools, which is how this case is avoided.

B. Resource limit reached (no more pools can be built)

Growth capped at 3:30: pool limit reached: 10 of POOL_MAX=10 pools

Not observed: the resource limit was reached, but every window diverted afterwards still found a free pool (a spare built before the limit), so nobody had to share.

Pool registry at the end of the run

As read from Redis just before the forced scale-down below.

Growth capped – pool limit reached: 10 of POOL_MAX=10 pools. New users reuse the existing pools round-robin instead of getting a new one.
Ready pools: 10 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10). Free (unowned) spares: 0. Users per pool: pool 1: 1, pool 2: 1, pool 3: 1, pool 4: 1, pool 5: 1, pool 6: 1, pool 7: 1, pool 8: 1, pool 9: 1, pool 10:

1. Round-robin reuse assignments so far: 0.

Scale-down

pool_manager scales down a pool once no request has reached it for POOL_IDLE_TIMEOUT_SECONDS = 1800 s (30:00): its users' assignments are dropped (a returning user is assigned afresh) and pools above the spare count (1) are removed with their volumes, freeing what they reserved. Compose-declared pools are only freed, never removed. Waiting that long is not practical in a test, so the run ends by forcing it: the windows stop sending requests, their sessions are released, and pool_manager removes the pools that are no longer needed at once. Other sessions' pools are not touched.

Requested at	5:50
Sessions released	10
Finished	after 16.7 s
Pools	10 → 3 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10 → 1, 2, 3); pools 1, 2, 3 are declared in docker-compose.yml, only freed, never removed, so more than the spare count (1) remain
Pools removed	pool 10 (1.5 s), pool 9 (1.4 s), pool 8 (1.7 s), pool 7 (1.6 s), pool 6 (1.5 s), pool 5 (1.6 s), pool 4 (1.2 s)
Reserved by container limits	7,680 MB / 7.5 CPUs → 2,304 MB / 2.25 CPUs
Memory in use (measured, pool containers)	4,144 MB → 1,345 MB (-2,799 MB)
CPU in use (measured, pool containers)	1.8 % → 0.0 % of all 20 CPUs (37 % → 0 % of one CPU)
Containers running	20 → 6
After the reassignment check	pool_manager started building pool(s) 4, 5 for the sessions the reassignment check (below) parked on a surviving pool. The test released those sessions again and removed what was built for them (second scale-down), leaving 3 pool(s): 1, 2, 3.

Reassignment after removal

A removed pool's owner keeps its session cookie; its pool assignment was dropped when the pool went. Sent back to the shop once, a session that was diverted should be routed to a surviving pool (shared round-robin if need be), keep its session and risk, and get a working page – not a 502 from the removed pool's container, and not production. The table records what the window actually got: its session before and after, the HTTP status of the page it loaded, and where it was routed. (The forced scale-down drops assignments on purpose; in normal operation idle scale-down only removes pools no request has reached, so there is usually no active owner to move.)

Window	Removed pool	Routed to	Session	HTTP	Outcome
User D	pool 4	pool 3	05f8d812… → same	200	**reassigned** moved to surviving pool 3, same session
User E	pool 6	pool 2	d9bfa301… → same	200	**reassigned** moved to surviving pool 2, same session
User F	pool 7	pool 3	d565d273… → same	200	**reassigned** moved to surviving pool 3, same session
User G	pool 5	pool 1	0c025b80… → same	200	**reassigned** moved to surviving pool 1, same session
User H	pool 8	pool 2	ef59993d… → same	200	**reassigned** moved to surviving pool 2, same session
User I	pool 10	pool 3	2c623349… → same	200	**reassigned** moved to surviving pool 3, same session
User J	pool 9	pool 1	de9049ee… → same	200	**reassigned** moved to surviving pool 1, same session

Every window whose pool was removed was moved to a surviving pool with its session, and its page loaded.

Cart persistence

A visitor diverted to a honeypot is served by a different database, so WooCommerce's own cart session does not exist there. The storefront therefore carries the cart in a cookie and rebuilds it on whichever instance serves the next request. Each window filled its cart on production; the cart was then read back after every step, through the same cookies and routing as the window's own traffic.

Where a window cleared its cookies (marked ◆), WooCommerce's cart cookie went with them, so the cart it filled is gone from the browser from that step on and an empty cart there is expected. If a cart shows up again after that, the browser did not bring it back: the stack restored it, and the report notes it below the table. That restored cart is then the one the later steps are compared with. Clearing only localStorage or sessionStorage leaves the cart cookie in place, so the cart must stay as it was.

The cart does not come back even when the stack recognises the window again and gives it its old session (fingerprint recovery, see Session tamper-proofing). That is by design: the session decides *where* a visitor is routed, while the cart lives only in the browser's *fh_cart* cookie, so that every instance – production or any honeypot pool, each with its own database – can rebuild it without sharing state. The server keeps no copy of a cart, so a visitor who deletes their cookies loses it, exactly as on a shop without a honeypot.

Rows are the steps of *Step by step*, columns the windows (A = User A, …). ✓ same cart as filled, ✗ cart changed, ○ empty since its cookies were cleared, ↻ cart back after clearing, – no cart yet, ? unreadable; ◆ cookies cleared before this step. Under each mark: where the window was served (P3 = honeypot pool 3, prod = production).

Step	A	B	C	D	E	F	G	H	I	J
1	– prod	– prod	– prod	– prod	– prod	– prod	– prod	– prod	– prod	– prod
2	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod
3	✓ P1	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod
4	✓ P1	✓ P2	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod
5	✓ P1	✓ P2	✓ P3	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod
6	✓ P1	✓ P2	✓ P3	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod
7	✓ P1	✓ P2	✓ P3	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod	✓ prod
8	✓ P1	✓ P2	✓ P3	✓ P4	✓ P6	✓ P7	✓ P5	✓ P8	✓ P10	○◆ P9
9	✓ P1	✓ P2	✓ P3	✓ P4	✓ P6	✓ P7	✓ P5	✓ P8	✓ P10	○ P9
10	✓ P1	✓ P2	✓ P3	○◆ P4	✓ P6	✓ P7	✓ P5	✓ P8	✓ P10	○ P9
11	✓ P1	✓ P2	✓ P3	○ P4	✓ P6	✓ P7	✓ P5	✓ P8	✓ P10	○ P9

Every cart stayed exactly as filled, on production and in every honeypot pool, until its window cleared its cookies; from then on it held only what the window had after clearing.

Patterns run per window

Each window runs a different pattern, picked from the ones a live probe showed divert within three attempts. If a chosen pattern still did not divert in three attempts, the next candidate is used instead.

Round	Window	Pattern	Name	Routing	Pool
1	A	CVE-2024-2387	Advanced Form Integration SQL injection	diverted after 1	pool 1
1	B	CVE-2025-47577	Gift Voucher file upload RCE	diverted after 1	pool 2
1	C	GENERIC-WP-003	User enumeration via ?author=	diverted after 1	pool 3
1	D	CVE-2024-27956	WP Automatic SQL injection	diverted after 1	pool 4
1	E	CVE-2022-0739	BookingPress SQL injection	diverted after 1	pool 6
1	F	CVE-2024-2879	LayerSlider SQL injection	diverted after 1	pool 7

Round	Window	Pattern	Name	Routing	Pool
1	G	CVE-2023-2986	Abandoned Cart Lite checkout token forgery	diverted after 1	pool 5
1	H	CVE-2025-2266	CWMP unauthenticated options update	diverted after 1	pool 8
1	I	GENERIC-SQLI-001	SQL injection in a search form	diverted after 2	pool 10
1	J	GENERIC-XSS-001	Script tag in a search form	diverted after 1	pool 9
2	A	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 1
2	B	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 2
2	C	GENERIC-WP-002	Installer file access	diverted after 1	pool 3
2	D	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 4
2	E	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 6
2	F	GENERIC-WP-002	Installer file access	diverted after 1	pool 7
2	G	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 5
2	H	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 8
2	I	GENERIC-WP-002	Installer file access	diverted after 1	pool 10
2	J	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 9

Step by step

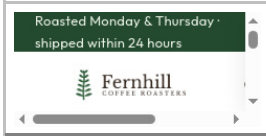
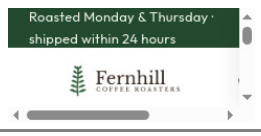
All windows move in every step. A user's pool must appear in their own step, differ from the earlier ones, and never change afterwards – except once, from a pool it only borrowed while its own was being built to that new pool.

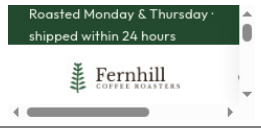
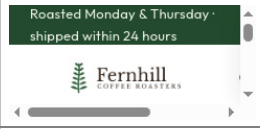
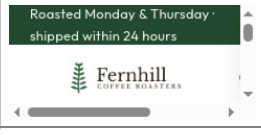
Under each routing badge: the session's pool, its risk score and, once diverted, why the router diverted it. A CVE pattern match or a request for a vulnerable plugin diverts the session whatever its score, so a diverted window can show a score below the diversion threshold (80 by default); only “score over the threshold” depends on it. The router

checks the score before its CVE list, so a pattern request that scores 100 by itself is recorded as “score over the threshold”; the pattern that did it is named next to it. The score adds up over the session's requests (capped at 100) and slowly fades between them, so one diverted window may show 94 where another shows 100 until its next suspicious request; once diverted, the session stays bound to its pool whatever its score does.

Step 1: All windows open the shop

10 new sessions, all routed to production.

User A	User B	User C	User D	User E
Browses the shop	Browses the shop	Browses the shop	Browses the shop	Browses the shop
production no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0
cart empty	cart empty	cart empty	cart empty	cart empty
				

User F	User G	User H	User I	User J
Browses the shop	Browses the shop	Browses the shop	Browses the shop	Browses the shop
production no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0
cart empty	cart empty	cart empty	cart empty	cart empty
				

Step 2: Every window fills its cart

Each window adds its own products. These carts are the baseline: they must look exactly the same after the session is routed to a honeypot pool.

User A	User B	User C	User D	User E
Adds ceramic-pour-over-dripper × 1, paper-filters-size-02 × 2	Adds hand-burr-coffee-grinder × 1, stainless-milk-pitcher × 1	Adds enamel-camp-mug × 2, fernhill-canvas-tote × 1	Adds gooseneck-electric-kettle × 1, logo-cap × 2	Adds digital-brew-scale × 1, then looks at Our Story
production no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Adds glass-french-press × 1, barista-apron × 1	Adds glass-bean-canister × 2, then looks at Shipping & Returns	Adds double-wall-glass-cups × 2, then looks at the FAQ	Adds insulated-travel-tumbler × 1, then looks at Coffee Club Subscription	Adds coffee-lover-gift-box × 1, pour-over-starter-kit × 1
production no pool, score 0	production no pool, score 0	production no pool, score 0	production no pool, score 0	production no pool, score 0
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	**$169.00** Coffee Lover Gift Box × 1 Pour-Over Starter Kit × 1
				

Step 3: User A attacks with CVE-2024-2387

Advanced Form Integration SQL injection. The session must be bound to a pool no other window has -- or, when no pool is free yet, borrow one round-robin until the pool being built for it is ready -- and the windows that have not attacked yet must stay on production.

User A	User B	User C	User D	User E
Opens CVE-2024-2387 (1 attempt(s)) page answered HTTP 404	Looks at Glass French Press	Looks at Drinkware	Looks at the Privacy Policy	◆ forged storage → ignored; then looks at Sunrise Espresso
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	production no pool, score 0	production no pool, score 0	production no pool, score 0	production no pool, score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Looks at the Terms of Service	Looks at Ethiopia Guji Natural	Looks at Stoneware Mug	Looks at Coffee Beans	Looks at Sunrise Espresso
production no pool, score 0	production no pool, score 0	production no pool, score 0	production no pool, score 0	production no pool, score 0
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	**$169.00** Coffee Lover Gift Box × 1 Pour-Over Starter Kit × 1
				

Step 4: User B attacks with CVE-2025-47577

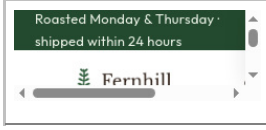
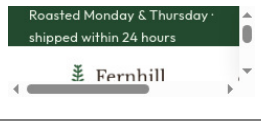
Gift Voucher file upload RCE. The session must be bound to a pool no other window has -- or, when no pool is free yet, borrow one round-robin until the pool being built for it is ready -- and the windows that have not attacked yet must stay on production.

User A	User B	User C	User D	User E
Looks at Kenya Nyeri AA	Opens CVE-2025-47577 (1 attempt(s)) page answered HTTP 404	Looks at the FAQ	Looks at Coffee Club Subscription	Looks at Kenya Nyeri AA
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	**honeypot** pool 2, score 95 why: score over the threshold, from its CVE-2025-47577 request	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Looks at Brewing Gear	Looks at Drinkware	Looks at the Privacy Policy	Looks at Our Story	Looks at Kenya Nyeri AA
production no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	**$169.00** Coffee Lover Gift Box × 1 Pour-Over Starter Kit × 1
				

Step 5: User C attacks with GENERIC-WP-003

User enumeration via ?author=. The session must be bound to a pool no other window has -- or, when no pool is free yet, borrow one round-robin until the pool being built for it is ready -- and the windows that have not attacked yet must stay on production.

User A	User B	User C	User D	User E
Looks at the Terms of Service	Looks at Shipping & Returns	Opens GENERIC-WP-003 (1 attempt(s))	Looks at Coffee Beans	Looks at the Terms of Service
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	honeypot pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	honeypot pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	production no pool, score 0	production no pool, score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Looks at Glass French Press	◆ hijack attempt → rejected; then looks at the FAQ	Looks at Coffee Club Subscription	Looks at Sunrise Espresso	Looks at the Terms of Service
production no pool, score 0	production no pool, score 50	production no pool, score 0	production no pool, score 0	production no pool, score 0
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	**$169.00** Coffee Lover Gift Box × 1 Pour-Over Starter Kit × 1
				

Step 6: Honeypot pools scale up

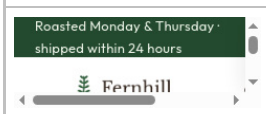
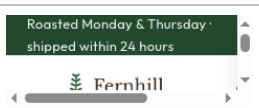
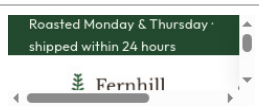
Windows that found no free pool (not scaled up yet) borrowed a ready one round-robin while pool_manager built a pool (eshop + seeded database) for each of them, and must now each be in their own. Once the resource limit is reached no more pools are built and the remaining windows keep sharing the ready pools round-robin.

User A	User B	User C	User D	User E
Looks at Brewing Gear	Looks at Ethiopia Guji Natural	Looks at Stoneware Mug	Looks at Our Story	Looks at Brewing Gear
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	honeypot pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	honeypot pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	production no pool, score 0	production no pool, score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Looks at Shipping & Returns	Looks at Stoneware Mug	Looks at Coffee Beans	Looks at Kenya Nyeri AA	Looks at Brewing Gear
production no pool, score 0	production no pool, score 50	production no pool, score 0	production no pool, score 0	production no pool, score 0
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	**$169.00** Coffee Lover Gift Box × 1 Pour-Over Starter Kit × 1
				

Step 7: Pools pre-built for 7 delayed window(s)

Before windows D, E, F, G, H, I, J attack, the test asks pool_manager for 7 free pool(s) and waits until they are ready, so these windows find a pool of their own the moment they are diverted instead of borrowing one round-robin. If the resource limit stops the build first, they share the ready pools round-robin instead. Outcome: 7 free pool(s) ready after 185 s.

User A	User B	User C	User D	User E
Looks at Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service	Looks at Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service, Brewing Gear, Glass French Press, Shipping & Returns	◆ forged cookie → rejected; then looks at the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service, Brewing Gear, Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ	Looks at Sunrise Espresso, Kenya Nyeri AA, the Terms of Service, Brewing Gear, Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans	Looks at Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	**honeypot** pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	**honeypot** pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	**production** no pool, score 0	**production** no pool, score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Looks at Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service, Brewing Gear, Glass French Press	Looks at the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service, Brewing Gear, Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ	Looks at Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service, Brewing Gear, Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription	Looks at the Terms of Service, Brewing Gear, Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso	Looks at Glass French Press, Shipping & Returns, Ethiopia Guji Natural, Drinkware, the FAQ, Stoneware Mug, the Privacy Policy, Coffee Club Subscription, Coffee Beans, Our Story, Sunrise Espresso, Kenya Nyeri AA, the Terms of Service
production no pool, score 0	**production** no pool, score 50	**production** no pool, score 0	**production** no pool, score 0	**production** no pool, score 0
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	**$169.00** Coffee Lover Gift Box × 1 Pour-Over Starter Kit × 1
				

Step 8: Delayed windows D, E, F, G, H, I, J attack together

These windows attack only now that pools were built for them: each must get a free pool of its own at once (no borrowing) -- unless the resource limit stopped the build, in which case they share the ready pools round-robin.

User A	User B	User C	User D	User E
Looks at Brewing Gear, Glass French Press	Looks at Ethiopia Guji Natural, Drinkware	Looks at Stoneware Mug, the Privacy Policy	Opens CVE-2024-27956 (1 attempt(s))	Opens CVE-2022-0739 (1 attempt(s))
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	honeypot pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	honeypot pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	honeypot pool 4, score 100 why: score over the threshold, from its CVE-2024-27956 request	honeypot pool 6, score 100 why: score over the threshold, from its CVE-2022-0739 request
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Opens CVE-2024-2879 (1 attempt(s))	Opens CVE-2023-2986 (1 attempt(s))	Opens CVE-2025-2266 (1 attempt(s))	Opens GENERIC-SQLI-001 (2 attempt(s))	◆ cleared site data → kept its session; then opens GENERIC-XSS-001 (1 attempt(s))
honeypot pool 7, score 100 why: score over the threshold, from its CVE-2024-2879 request	honeypot pool 5, score 92 why: score over the threshold, from its CVE-2023-2986 request	honeypot pool 8, score 94 why: CVE pattern match	honeypot pool 10, score 100 why: score over the threshold, from its GENERIC-SQLI-001 request	honeypot pool 9, score 100 why: score over the threshold, from its GENERIC-XSS-001 request
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	cart empty
				

Step 9: Honeypot pools scale up

Windows that found no free pool (not scaled up yet) borrowed a ready one round-robin while pool_manager built a pool (eshop + seeded database) for each of them, and must now each be in their own. Once the resource limit is reached no more pools are built and the remaining windows keep sharing the ready pools round-robin.

User A	User B	User C	User D	User E
Looks at Shipping & Returns	Looks at the FAQ	Looks at Coffee Club Subscription	Looks at Our Story	Looks at Brewing Gear
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	honeypot pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	honeypot pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	honeypot pool 4, score 100 why: score over the threshold, from its CVE-2024-27956 request	honeypot pool 6, score 100 why: score over the threshold, from its CVE-2022-0739 request
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Looks at Shipping & Returns	Looks at Stoneware Mug	Looks at Coffee Beans	Looks at Kenya Nyeri AA	Looks at Brewing Gear
honeypot pool 7, score 100 why: score over the threshold, from its CVE-2024-2879 request	honeypot pool 5, score 92 why: score over the threshold, from its CVE-2023-2986 request	honeypot pool 8, score 94 why: CVE pattern match	honeypot pool 10, score 100 why: score over the threshold, from its GENERIC-SQLI-001 request	honeypot pool 9, score 100 why: score over the threshold, from its GENERIC-XSS-001 request
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	cart empty
				

Step 10: All windows run a second pattern

Sticky binding: each second pattern must land in the pool the window already owns, nobody changes pool, nobody is merged.

User A	User B	User C	User D	User E
Opens CVE-2024-1071 page answered HTTP 400	Opens CVE-2025-4403 page answered HTTP 404	Opens GENERIC-WP-002	◆ cleared site data → kept its session; then opens CVE-2024-1071 page answered HTTP 400	Opens CVE-2025-4403 page answered HTTP 404
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	honeypot pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	honeypot pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	honeypot pool 4, score 100 why: score over the threshold, from its CVE-2024-27956 request	honeypot pool 6, score 100 why: score over the threshold, from its CVE-2022-0739 request
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	cart empty	**$49.00** Digital Brew Scale with Timer × 1
0			0	

User F	User G	User H	User I	User J
Opens GENERIC-WP-002	Opens CVE-2024-1071 page answered HTTP 400	Opens CVE-2025-4403 page answered HTTP 404	Opens GENERIC-WP-002	Opens CVE-2024-1071 page answered HTTP 400
honeypot pool 7, score 100 why: score over the threshold, from its CVE-2024-2879 request	honeypot pool 5, score 100 why: score over the threshold, from its CVE-2023-2986 request	honeypot pool 8, score 100 why: CVE pattern match	honeypot pool 10, score 100 why: score over the threshold, from its GENERIC-SQLI-001 request	honeypot pool 9, score 100 why: score over the threshold, from its GENERIC-XSS-001 request
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	cart empty
	0			0

Step 11: Every window opens its cart

The cart page, served by each window's honeypot pool, must show the same products and quantities that were added on production.

User A	User B	User C	User D	User E
◆ forged cookie → rejected; then opens the cart page	Opens the cart page	Opens the cart page	Opens the cart page	Opens the cart page
honeypot pool 1, score 100 why: score over the threshold, from its CVE-2024-2387 request	honeypot pool 2, score 100 why: score over the threshold, from its CVE-2025-47577 request	honeypot pool 3, score 100 why: score over the threshold, from its GENERIC-WP-003 request	honeypot pool 4, score 100 why: score over the threshold, from its CVE-2024-27956 request	honeypot pool 6, score 100 why: score over the threshold, from its CVE-2022-0739 request
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	cart empty	**$49.00** Digital Brew Scale with Timer × 1
				

User F	User G	User H	User I	User J
Opens the cart page	Opens the cart page	Opens the cart page	Opens the cart page	Opens the cart page
honeypot pool 7, score 100 why: score over the threshold, from its CVE-2024-2879 request	honeypot pool 5, score 100 why: score over the threshold, from its CVE-2023-2986 request	honeypot pool 8, score 100 why: CVE pattern match	honeypot pool 10, score 100 why: score over the threshold, from its GENERIC-SQLI-001 request	honeypot pool 9, score 100 why: score over the threshold, from its GENERIC-XSS-001 request
$90.00 Glass French Press 34 oz × 1 Barista Apron × 1	**$49.95** Glass Bean Canister × 2	**$52.00** Double-Wall Glass Cups (Set of 2) × 2	**$34.95** Insulated Travel Tumbler 16 oz × 1	cart empty
				

What each honeypot container saw and changed

Read from each pool's own eshop and database containers since the run started (2026-10-17T14:33:09Z). Requests are matched to windows by their User-Agent tag, so a pool should show its owner's patterns and nobody else's. A pool no window owns is shown for contrast.

pool 1 window A

Windows seen	A (33 requests)
Requests received	• **A** GET /?page=advanced-form-integration-log&integration_id=1%27%20UNION%20SELECT%20user_login%2Cu → 404 • **A** GET /wp-json/wc/store/v1/cart → 200 • **A** GET /product/kenya-nyeri-aa/ → 200 • **A** GET /terms-of-service/ → 200 • **A** GET /product-category/brewing-gear/ → 200 • **A** GET /product/ceramic-pour-over-dripper/ → 200 • **A** GET /product/glass-french-press/ → 200 • **A** GET /shipping-returns/ → 200
Files changed	• no files written
Database activity	**questions +4829, select +4650, insert +20, update +14, delete +12** beyond its own background total questions +10310, select +9740, insert +54, update +36, delete +46; the background (healthchecks, WP cron, content sync) was measured while every window was still on production

pool 2 window B

Windows seen	B (31 requests)
Requests received	• **B** GET /?action=mwb_wgm_preview_mail&attachment=shell.php → 404 • **B** GET /wp-json/wc/store/v1/cart → 200 • **B** GET /shipping-returns/ → 200 • **B** GET /product/ethiopia-guji-natural/ → 200 • **B** GET /product/ceramic-pour-over-dripper/ → 200 • **B** GET /product-category/drinkware/ → 200 • **B** GET /faq/ → 200 • **B** GET /product/stoneware-mug/ → 200
Files changed	• no files written
Database activity	**questions +4473, select +4316, insert +19, update +8, delete +12** beyond its own background total questions +9968, select +9418, insert +53, update +30, delete +46; the background (healthchecks, WP cron, content sync) was measured while every window was still on production

pool 3 window C

Windows seen	C (30 requests)
Requests received	• C GET /?author=1 → 301 • C GET /author/admin/ → 200 • C GET /wp-json/wc/store/v1/cart → 200 • C GET /product/stoneware-mug/ → 200 • C GET /product/ceramic-pour-over-dripper/ → 200 • C GET /privacy-policy/ → 200 • C GET /product/coffee-club-subscription/ → 200 • C GET /product-category/coffee/ → 200
Files changed	• no files written
Database activity	**questions +4232, select +4094, insert +14, update +9, delete +8** beyond its own background total questions +9742, select +9210, insert +48, update +31, delete +42; the background (healthchecks, WP cron, content sync) was measured while every window was still on production

pool 4 window D

Windows seen	D (10 requests)
Requests received	• D GET /wp-content/plugins/wp-automatic/inc/csv.php?q=1%27%20UNION%20SELECT%20user_login%2Cuser_p → 404 • D GET / → 200 • D GET /wp-json/wc/store/v1/cart → 200 • D GET /about/ → 200 • D GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400 • D GET /product/ceramic-pour-over-dripper/ → 200 • D GET /cart/ → 200
Files changed	• no files written
Database activity	questions +5379, select +4944, insert +62, update +70, delete +34 total, including the pool's own background (not measured for a pool that came up during the run)

pool 5 window G

Windows seen	G (10 requests)
Requests received	• G GET /?wcal_action=checkout_link&validate=x → 200 • G GET / → 200 • G GET /wp-json/wc/store/v1/cart → 200 • G GET /product/stoneware-mug/ → 200 • G GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400 • G GET /product/ceramic-pour-over-dripper/ → 200 • G GET /cart/ → 200
Files changed	• no files written
Database activity	questions +5239, select +4812, insert +69, update +72, delete +33 total, including the pool's own background (not measured for a pool that came up during the run)

pool 6 window E

Windows seen	E (10 requests)
Requests received	• E GET /wp-admin/admin-ajax.php?action=bookingpress_front_get_category_services&total_service=1%2 → 400 • E GET / → 200 • E GET /wp-json/wc/store/v1/cart → 200 • E GET /product-category/brewing-gear/ → 200 • E GET /?action=dnd-wc-upload-file&name=shell.php → 404 • E GET /product/ceramic-pour-over-dripper/ → 200 • E GET /cart/ → 200
Files changed	• no files written
Database activity	questions +4522, select +4140, insert +60, update +70, delete +29 total, including the pool's own background (not measured for a pool that came up during the run)

pool 7 window F

Windows seen	F (10 requests)
Requests received	• F GET /wp-admin/admin-ajax.php?action=ls_get_popup_markup&id=1%27%20UNION%20SELECT%20user_login% → 400 • F GET / → 200 • F GET /wp-json/wc/store/v1/cart → 200 • F GET /shipping-returns/ → 200 • F GET /wp-admin/install.php → 200 • F GET /product/ceramic-pour-over-dripper/ → 200 • F GET /cart/ → 200
Files changed	• no files written
Database activity	questions +3987, select +3655, insert +56, update +68, delete +21 total, including the pool's own background (not measured for a pool that came up during the run)

pool 8 window H

Windows seen	H (10 requests)
Requests received	• H GET /?action=cwmpUpdateOptions&option=default_role&value=administrator → 200 • H GET / → 200 • H GET /wp-json/wc/store/v1/cart → 200 • H GET /product-category/coffee/ → 200 • H GET /?action=dnd-wc-upload-file&name=shell.php → 404 • H GET /product/ceramic-pour-over-dripper/ → 200 • H GET /cart/ → 200
Files changed	• no files written
Database activity	questions +3617, select +3378, insert +56, update +26, delete +14 total, including the pool's own background (not measured for a pool that came up during the run)

pool 9 window J

Windows seen	J (11 requests)
Requests received	• J GET /?s=%3Cscript%3Ealert(1)%3C/script%3E → 200 • J GET / → 200 • J GET /wp-json/wc/store/v1/cart → 200 • J GET /product-category/brewing-gear/ → 200 • J GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400 • J GET /product/ceramic-pour-over-dripper/ → 200 • J GET /cart/ → 200 • J POST /wp-json/wc/store/v1/batch?_locale=site → 207
Files changed	• no files written
Database activity	questions +3267, select +2974, insert +48, update +68, delete +19 total, including the pool's own background (not measured for a pool that came up during the run)

pool 10 window I

Windows seen	I (9 requests)
Requests received	• I GET /?s=1%27%20OR%20%271%27=%271 → 200 • I GET /wp-json/wc/store/v1/cart → 200 • I GET /product/kenya-nyeri-aa/ → 200 • I GET /wp-admin/install.php → 200 • I GET /product/ceramic-pour-over-dripper/ → 200 • I GET /cart/ → 200
Files changed	• no files written
Database activity	questions +2447, select +2311, insert +32, update +6, delete +6 total, including the pool's own background (not measured for a pool that came up during the run)